# Baza ta de date time-series în C

Ghid actualizat: jurnalul WAL, recuperarea sigură, indexul și comenzile CLI

**Ce avem:** record binar de 92 de octeți, loturi confirmate în WAL, index sortat în memorie, căutări pe interval și count/min/max/avg. Fișierele noi folosesc un format WAL v2, cu antetul BEGIN verificat independent.

**De ce am schimbat recuperarea:** vechiul cod putea confunda un câmp **count** deteriorat cu o scriere neterminată și putea tăia date confirmate. Deschiderea actuală nu mai trunchiază nimic. Repararea este o comandă separată, cu backup obligatoriu.

```
punct -> BEGIN + RECORD + COMMIT -> WAL pe disc
                                      |
                                replay validat
                                      |
                           index sortat in RAM
                                      |
                             query / aggregate
```

**Starea verificată:** build MSVC reușit, 6/6 teste CTest trecute. CLI-ul a fost verificat separat pentru fișier lipsă, init repetat, scriere și interogare. Segmentele pe disc și compactarea nu sunt încă implementate.

Actualizat: 6 octombrie 2026.

# 1. Formatul de pe disc și CRC-ul

Un **TsdbPoint** conține un nume de metrică, un timestamp int64 în milisecunde și un double finit. Funcția **tsdb_record_encode** scrie exact 92 de octeți, independent de paddingul structurii C.

| Octeți | Recordul conține |
|---|---|
| 0-7 | Magic TSR1, versiune, lungimea metricii și câmpuri rezervate |
| 8-15 | Timestamp int64, little-endian |
| 16-23 | Biții valorii double, little-endian |
| 24-87 | Nume metrică și completare cu zero |
| 88-91 | CRC-32 al octeților 0-87 |

## WAL v2: formatul folosit de fișierele noi

```
FILE_HEADER(16)
BEGIN v2(20) | RECORD(92) ... | COMMIT(16)
BEGIN v2 = magic + count + sequence + CRC propriu
```

CRC-ul din BEGIN v2 acoperă magic, count și sequence. Recuperarea verifică acești octeți **înainte** să folosească count pentru a calcula lungimea lotului. COMMIT păstrează un alt CRC peste BEGIN și toate recordurile. CRC detectează erori accidentale; nu este un mecanism de securitate criptografică.

## Ce se întâmplă cu fișierele vechi?

WAL v1 începe direct cu **BEGIN(16)**, fără antet de fișier și fără CRC propriu pentru BEGIN. Codul actual îl poate citi și poate continua scrierea în v1. Dacă BEGIN-ul v1 pare incomplet, nu îl șterge automat; situația poate fi ambiguă. Un v1 fără niciun lot valid este refuzat de repararea automată.

## 2. Deschidere, scriere și reparare

| Acțiune | Ce face | Când o folosești |
|---|---|---|
| init | Creează numai un WAL v2 nou. Refuză un fișier existent. | O singură dată, la început. |
| write | Deschide un WAL existent pentru scriere; confirmă lotul după FlushFileBuffers. | Când adaugi un punct. |
| query / aggregate | Deschide doar pentru citire. Nu creează și nu trunchiază fișierul. | Când citești datele. |
| repair | Copiază întregul WAL într-un backup nou, apoi taie numai coada incompletă. | După TSDB_ERR_INCOMPLETE și inspecția situației. |

### Exemplul care a motivat schimbarea

Două loturi v1 valide cu câte un punct ocupă 248 de octeți. Dacă **count** din primul BEGIN se schimbă din 1 în 3, vechiul cititor aștepta 308 octeți și putea tăia tot fișierul la zero. Noul cititor v1 raportează coadă incompletă **fără modificarea fișierului**. Pentru v2, același bit schimbat este prins direct de CRC-ul din BEGIN.

### Ce face FlushFileBuffers?

Este un **apel Windows din codul C**, nu o comandă de terminal. După WriteFile, cere sistemului de operare să sincronizeze datele. Biblioteca raportează succes pentru write_batch numai după acest apel. Nu există garanție absolută împotriva oricărei defecțiuni hardware.

• Dacă backupul există deja, repair îl refuză și nu trunchiază sursa.

• Dacă un lot complet are CRC greșit, deschiderea întoarce eroare și păstrează fișierul.

• Dacă o scriere întoarce eroare I/O, închide și redeschide înainte de a decide ce reîncerci.

## 3. Indexul și interogările

După deschidere, replay citește loturile complete și construiește un tablou dinamic ordonat după **(metrică, timestamp)**. Un punct poate sosi târziu; indexul îl mută la poziția corectă. Pentru aceeași cheie, ultima scriere confirmată înlocuiește valoarea veche.

```
Scrieri:  cpu@200=20, cpu@100=10, cpu@200=25
Index:    cpu@100=10, cpu@200=25
```

**query_range** face două căutări binare și întoarce punctele din interval. În implementarea actuală, ambele capete sunt **incluse**. **aggregate** calculează count, min, max și avg. Pentru interval gol întoarce count=0 și zerouri pentru celelalte câmpuri.

Căutarea costă aproximativ O(log N + K) pentru K rezultate din N puncte. Inserarea poate muta O(N) elemente. La fiecare nouă deschidere, indexul este reconstruit din tot WAL-ul. De aceea această versiune este potrivită pentru învățare și volume moderate.

### De ce rezervăm memorie înainte să confirmăm WAL-ul?

**tsdb_write_batch** rezervă întâi spațiu în index, apoi scrie și sincronizează lotul în WAL, apoi îl face vizibil în RAM. Inserarea de după confirmare nu mai trebuie să aloce memorie. Astfel evităm situația în care datele sunt confirmate pe disc, dar nu pot fi adăugate în index din lipsă de memorie.

### Ce rămâne de decis înainte de segmente?

Indexul actual nu păstrează numărul commitului pentru fiecare cheie. Înainte de etapa cu segmente, vom adăuga informația necesară pentru a alege corect între o valoare din segment și una scrisă mai târziu.

## 4. Comenzile exacte și ce fac

| Comandă | Explicație |
|---|---|
| cmake -S . -B build | Configurează proiectul în folderul build; nu compilează. |
| cmake --build build --config Debug | Compilează biblioteca, CLI-ul și testele cu MSVC. |
| ctest --test-dir build -C Debug --output-on-failure | Rulează executabilele de test Debug și arată erorile. |
| tsdb.exe init date.bin | Creează date.bin; eșuează dacă există deja. |
| tsdb.exe write date.bin cpu 1000 42.5 | Adaugă un punct CPU și îl confirmă în WAL. |
| tsdb.exe query date.bin cpu 0 2000 | Afișează punctele CPU din intervalul inclusiv. |
| tsdb.exe aggregate date.bin cpu 0 2000 | Afișează count, min, max și avg. |
| tsdb.exe repair date.bin copie.bin | Creează copie.bin și apoi elimină o coadă incompletă. |

Rulează întâi **build**, apoi **CTest**. Dacă build-ul a eșuat, CTest poate rula executabile vechi. Rulează comenzile CLI din directorul proiectului, folosind calea **.\build\Debug\tsdb.exe**.

```
.\build\Debug\tsdb.exe init demo.bin
.\build\Debug\tsdb.exe write demo.bin cpu 100 20.5
.\build\Debug\tsdb.exe write demo.bin cpu 50 10
.\build\Debug\tsdb.exe query demo.bin cpu 0 200
# cpu 50 10; apoi cpu 100 20.5
.\build\Debug\tsdb.exe aggregate demo.bin cpu 0 200
# count=2 min=10 max=20.5 avg=15.25
```

**Limite:** un singur writer, indexul integral în RAM, inserări potențial lente, fără segmente sau compactare. Testele verifică cozi incomplete, CRC și compatibilitate v1, dar nu simulează încă toate erorile posibile ale discului.